Guion del video – Pedidos360

Evaluación 2 · DSY1107 Desarrollo Cloud Native I · Mensajería asíncrona con RabbitMQ

Video de **unos 9:50 minutos** en tres partes. Cada integrante graba la suya por separado (pantalla + voz) y Sebastián las une en este orden. La pauta pide entre 5 y 10 minutos.

Parte	Quién	Duración	Qué cubre	Archivo
1	Integrante 2	2:29 (grabada)	Problema, arquitectura, flujo de un pedido y diseño de seguridad (JWT en dos capas)	`parte1-integrante2.mp4`
2	Sebastián	máximo 3:00	Demo en vivo: Entra ID, AWS (EC2, RDS, API Gateway), login, pedido, rechazo de JWT, docker compose, clúster, exchanges, colas y DLQ	`parte2-sebastian.mp4`
3	Integrante 3	4:30 (grabada)	Código: configuración centralizada, beans por caso de uso, consumidores por dominio, ACK/NACK/DLQ, administrador REST con validación, pruebas y cierre	`parte3-integrante3.mp4`

Cómo grabar y unir

- **Grabar:** Win + Alt + R (grabador de Windows), OBS, o una reunión de Teams/Zoom grabada compartiendo pantalla. MP4, 1080p, micrófono cerca, sin música.
- **Pantalla:** navegador o PDF a pantalla completa, zoom 125 % para que se lea, notificaciones silenciadas.
- **Errores:** no repitan todo. Pausen dos segundos y retomen desde el inicio del bloque; el error se corta al unir.
- **Unir:** Clipchamp (viene con Windows): importar las tres partes, ponerlas en orden y exportar en 1080p.
- Integrante 2 e Integrante 3 **no necesitan** AWS, Azure ni el código instalado: todo lo que muestran está en este PDF. La demo en vivo es solo de Sebastián porque el laboratorio de AWS y el tenant de Entra ID están en su cuenta.

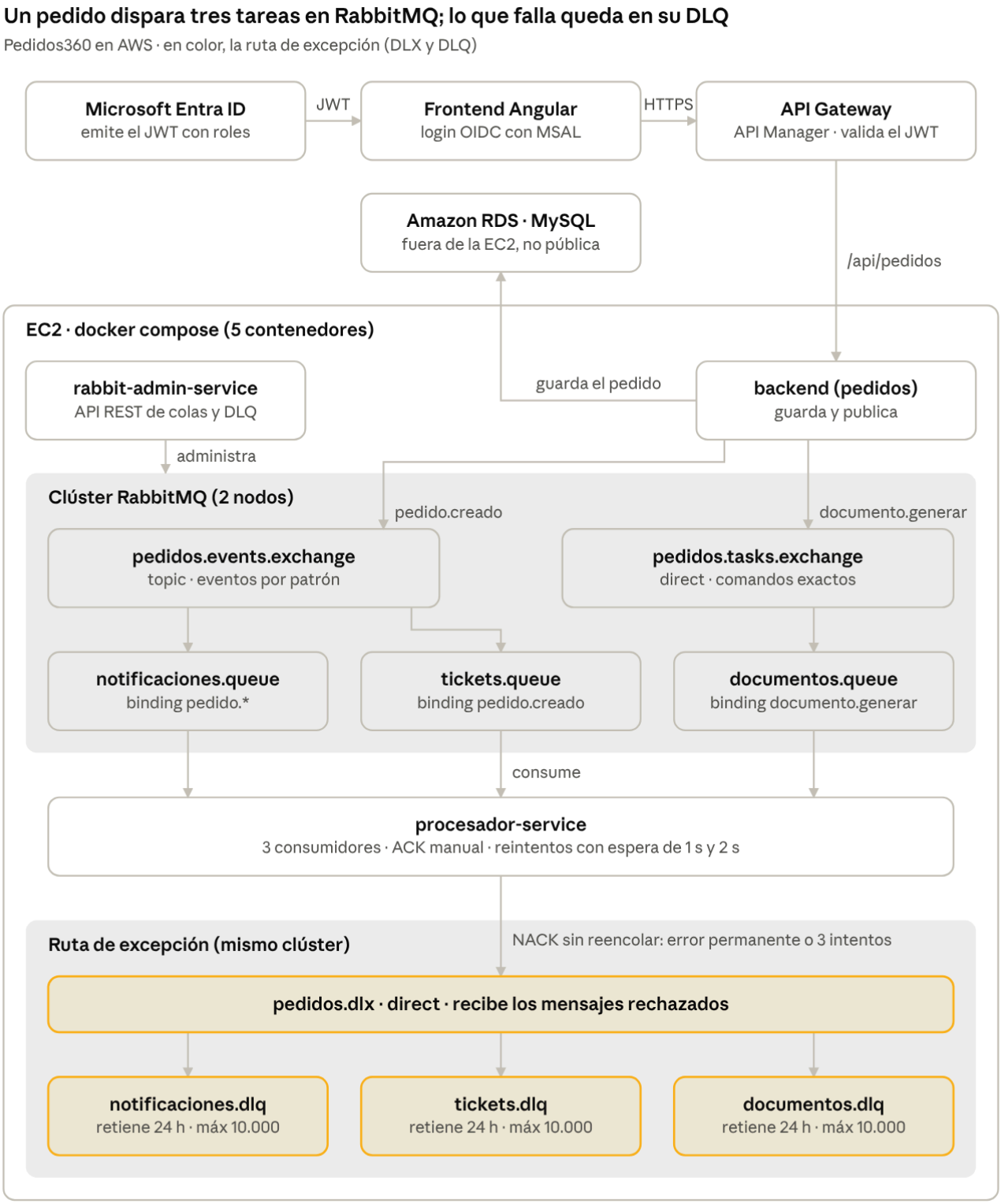
Requisitos de la pauta (EP4) y dónde se muestran

Requisito de la presentación	Dónde
1. Instancia del API Manager en la nube	Parte 2, toma 2

2. Configuración del API Manager hacia los endpoints del backend	Parte 2, toma 2
3. Frontend consume los endpoints a través del API Manager	Parte 2, toma 4
4. El API Manager valida JWT: rechaza inválidos y acepta correctos	Parte 2, tomas 4 y 5
5. Tenant en IDaaS y usuarios registrados	Parte 2, toma 1
6. Frontend usa OAuth 2.0 / OpenID Connect y obtiene un JWT válido	Parte 2, toma 3
7. Backend y frontend desplegados, activos e integrados en la nube	Parte 2, tomas 2 y 4
8. Dos nodos RabbitMQ en clúster	Parte 2, toma 7 · Parte 1, bloque 5
9. Tres colas con sus DLQ funcionando	Parte 2, tomas 7 y 8 · Parte 3, bloques 3 y 5
10. Exchanges de tipo direct y topic	Parte 2, toma 7 · Parte 1, bloque 3
11. docker-compose levantando RabbitMQ en la nube	Parte 2, toma 6

Arquitectura

El backend publica un evento (topic) y un comando (direct); el procesador consume las tres colas y lo que no puede procesar termina, vía el DLX, en la DLQ de su cola. Este diagrama es lo que muestran la Parte 1 y el bloque 1 de la Parte 3.



Parte 1 – Integrante 2: arquitectura y seguridad (2:00 – 2:30)

En pantalla: el diagrama de la página de Arquitectura, a pantalla completa. Señala con el mouse cada caja cuando la nombres.

1. Presentación · 15 s

Muestra: el diagrama completo.

> "Hola, somos el equipo de Pedidos360. Les vamos a mostrar cómo incorporamos mensajería asíncrona con RabbitMQ al sistema que construimos en la evaluación anterior. Yo explico la arquitectura, Sebastián muestra la demo en la nube y [nombre del Integrante 3] explica el código."

2. Punto de partida · 20 s

Señala: Microsoft Entra ID, Frontend Angular y API Gateway (fila de arriba) y Amazon RDS.

> "Pedidos360 ya tenía un frontend en Angular con inicio de sesión en Microsoft Entra ID, un backend en Spring Boot con base de datos MySQL en Amazon RDS, y un API Gateway en AWS como API Manager. Ahora, al crear un pedido, hay tareas que no deben hacer esperar al usuario: avisar al cliente, generar un ticket y generar el comprobante."

3. Flujo de un pedido · 45 s

Señala: backend → pedidos.events.exchange → notificaciones.queue y tickets.queue; luego pedidos.tasks.exchange → documentos.queue; al final procesador-service.

> "El usuario crea el pedido en Angular. El API Gateway valida el token y lo envía al backend, que guarda el pedido en RDS y publica dos mensajes en RabbitMQ. El primero es el evento `pedido.creado`, que va a un exchange de tipo **topic**: la cola de notificaciones está enlazada con el patrón `pedido.*`, así que recibe cualquier evento de pedido, y la cola de tickets solo con `pedido.creado`. El segundo es el comando `documento.generar`, que va a un exchange de tipo **direct** y llega solo a la cola de documentos, por coincidencia exacta. Un tercer microservicio, el procesador, consume las tres colas."

4. Qué pasa cuando algo falla · 30 s

Señala: procesador-service → pedidos.dlx → las tres DLQ (las cajas en color).

> "Si el procesamiento falla por algo transitorio, el procesador reintenta tres veces con espera creciente de 1 y 2 segundos. Si sigue fallando, o si el error es permanente como un dato inválido, rechaza el mensaje y RabbitMQ lo reenvía al Dead Letter Exchange, que lo deja en la DLQ de esa cola. Cada DLQ guarda los mensajes hasta 24 horas y máximo 10.000. Y si RabbitMQ estuviera caído, el pedido igual se guarda: la mensajería no rompe la lógica existente."

5. Administración e infraestructura · 20 s

Señala: rabbit-admin-service, el recuadro "Clúster RabbitMQ (2 nodos)" y el recuadro de la EC2.

> "Agregamos un microservicio administrador con una API REST para crear y eliminar colas, exchanges y bindings, y para monitorear las DLQ. RabbitMQ corre como un clúster de dos nodos levantado con docker compose en una instancia EC2."

6. Seguridad · 20 s

Señala: Microsoft Entra ID y API Gateway.

> "El token JWT de Entra ID se valida en dos capas: primero en el API Gateway, con un autorizador JWT que revisa el emisor y la audiencia; después en cada microservicio, que revisa firma, emisor, audiencia, vigencia y roles. Crear pedidos exige el rol Cliente y modificar RabbitMQ exige el rol Admin."

7. Cierre y traspaso · 5 s

> "Ahora Sebastián les muestra todo funcionando en la nube."

Parte 2 – Sebastián: demo en vivo en AWS (máximo 3:00)

Direcciones del despliegue del 25 de octubre de 2026. Si el laboratorio se reinicia, la IP de la EC2 y el API Gateway cambian (`infra/aws/README.md`).

Antes de grabar

Frontend	https://6iqkstpi9c.execute-api.us-east-1.amazonaws.com
Consola RabbitMQ	http://100.30.255.138:15672 · usuario y clave en `infra/aws/.env.cloud` (solo abre desde tu IP)
Decodificar el token	https://jwt.ms

☐ **Entra ID:** Pedidos360-Web → Authentication → "+ Agregar URI de redireccionamiento" → Aplicación de página única → `https://6iqkstpi9c.execute-api.us-east-1.amazonaws.com` (sin barra final). La URL anterior ya no existe.

☐ Inicia sesión una vez en el frontend y crea un pedido de prueba. Si da 403, asigna el rol Cliente en Enterprise applications → Pedidos360-API → Users and groups.

☐ Deja estas pestañas abiertas, en este orden: Entra ID (Información general) · EC2 · RDS · API Gateway `pedidos360-api` → Routes · frontend (con sesión iniciada) · jwt.ms · consola RabbitMQ (con sesión iniciada).

☐ Abre la terminal de la nube: EC2 → `pedidos360-ec2` → Conectar → Session Manager → Conectar.

☐ Abre PowerShell con este comando ya escrito (sin Enter): `curl.exe -i https://6iqkstpi9c.execute-api.us-east-1.amazonaws.com/api/pedidos`

☐ Las DLQ están en 0. Si haces pruebas antes y quedan mensajes: consola RabbitMQ → cada `*.dlq` → Purge Messages.

☐ Graba antes de las 17:52 hora de Chile (13:52 hora del lab, fin de la sesión). Si el lab se reinicia, la IP y el gateway cambian.

Tomas (2:50 en total)

1. Entra ID · 15 s `req. 5`

Información general (Id. de inquilino) → Usuarios (2) → Registros de aplicaciones (Pedidos360-API y Pedidos360-Web).

> "Este es nuestro tenant de Microsoft Entra ID, con dos usuarios y las aplicaciones del frontend y de la API, que define los roles Cliente y Admin."

2. Infraestructura en AWS · 25 s req. 1, 2 y 7

EC2 `pedidos360-ec2` en ejecución → RDS `pedidos360-db` disponible → API Gateway `pedidos360-api` → Routes (las rutas `/api/...` con JWT) → Authorization (autorizador `entra-jwt`).

> "En AWS tenemos la EC2 con RabbitMQ y los tres microservicios, la base de datos en RDS y el API Gateway como API Manager: cada ruta tiene un autorizador JWT con el emisor y la audiencia de Entra ID."

3. Login y JWT · 25 s req. 6

Frontend → Iniciar sesión (o muestra que ya estás dentro) → F12 → Network → petición `pedidos` → copia el header `Authorization` sin "Bearer" → pégalo en jwt.ms.

> "El frontend inicia sesión con OpenID Connect y PKCE. Entra ID entrega este JWT: emisor nuestro tenant, audiencia la API y el rol Cliente."

4. Pedido a través del API Manager · 15 s req. 3 y 4

Crea un pedido (Ana · Café 1 kg · 15990) → aparece en la tabla; en Network el POST va a `6iqkstpi9c.execute-api...` con 200.

> "El frontend llama al backend a través del API Gateway; con token válido la petición pasa y el pedido se guarda."

5. Rechazo sin token · 10 s req. 4

En PowerShell, Enter al comando ya escrito → `HTTP/1.1 401 Unauthorized`.

> "Sin un token válido, el API Gateway responde 401 y la petición no llega al backend."

6. docker compose en la nube · 15 s req. 11

En Session Manager:

```
cd /opt/pedidos360 && sudo docker compose ps
```

> "Todo corre con docker compose en la EC2: los dos nodos de RabbitMQ y los tres microservicios."

7. RabbitMQ: clúster, exchanges y colas · 30 s req. 8, 10 y 9

Overview → tabla Nodes (`rabbit@rabbit1` y `rabbit@rabbit2`) → Exchanges (`pedidos.events.exchange` topic, `pedidos.tasks.exchange` direct, `pedidos.dlx`) → Queues (3 colas + 3 DLQ, todas en 0).

> "RabbitMQ corre en un clúster de dos nodos. Tenemos un exchange topic para eventos, uno direct para comandos y el DLX. Cada una de las tres colas tiene su DLQ, con retención de 24 horas."

8. Una DLQ funcionando en vivo · 30 s req. 9

En el frontend crea un pedido con descripción `Prueba #FAIL-TICKET` → en Session Manager:

```
sudo docker logs --tail 6 pedidos360-procesador
→ intento 1/3 ... intento 2/3 ... NACK -> DLQ
```

→ en RabbitMQ, Queues: `tickets.dlq` con 1 mensaje.

> "Este pedido simula que el servicio de tickets está caído: el procesador reintenta tres veces y lo rechaza, y el mensaje queda en la DLQ de tickets. La notificación y el documento del mismo pedido sí se procesaron."

9. Traspaso · 5 s

> "Ahora [nombre del Integrante 3] explica cómo está hecho esto en el código."

Parte 3 – Integrante 3: código de mensajería (2:00 – 2:30)

En pantalla: bloque 1, el diagrama de Arquitectura; bloques 2 a 6, el fragmento de código de cada bloque (copias recortadas del código real). Si el repositorio está en GitHub, se pueden mostrar ahí los mismos archivos; el texto no cambia.

1. Estructura · 15 s

Muestra: el diagrama. Señala backend, procesador-service y rabbit-admin-service.

> "El backend son tres microservicios Spring Boot: `backend`, que guarda el pedido y publica los mensajes; `procesador-service`, con los consumidores; y `rabbit-admin-service`, que administra RabbitMQ por API REST."

2. Nombres centralizados · 25 s

Muestra: `procesador-service/src/main/resources/application.yml`

```yaml
pedidos360:
  messaging:
    exchanges:
      events: pedidos.events.exchange        # TOPIC
      tasks: pedidos.tasks.exchange          # DIRECT
      dead-letter: pedidos.dlx               # DIRECT - DLX común
    queues:
      tickets:
        name: tickets.queue
        binding-key: pedido.creado
        dlq: tickets.dlq
        dead-letter-key: tickets.dead
    dead-letter:                             # retención de las DLQ
      retention-ms: 86400000                 # 24 h
      max-length: 10000
    retry:
      max-attempts: 3
      initial-interval-ms: 1000
      multiplier: 2.0
```

> "Todos los nombres de colas, exchanges y bindings viven en el application.yml de cada servicio y se leen con `@ConfigurationProperties`. Ninguna clase escribe un nombre a mano."

3. Beans por caso de uso · 25 s

Muestra: `config/TicketsTopologyConfig.java` (hay una igual para notificaciones y otra para documentos).

```
@Configuration
public class TicketsTopologyConfig {
    // cola principal, con x-dead-letter-exchange
    @Bean Queue ticketsQueue(MessagingProperties props) {
        return TopologySupport.mainQueue(props.queues().tickets(), props);
    }
    // DLQ, con TTL y max-length (retención)
    @Bean Queue ticketsDlq(MessagingProperties props) {
        return TopologySupport.deadLetterQueue(props.queues().tickets(), props);
    }
    @Bean Binding ticketsBinding(@Qualifier("ticketsQueue") Queue queue,
            TopicExchange eventsExchange, MessagingProperties props) {
        return BindingBuilder.bind(queue).to(eventsExchange)
                .with(props.queues().tickets().bindingKey());
    }
    @Bean Binding ticketsDlqBinding(@Qualifier("ticketsDlq") Queue dlq,
            @Qualifier("deadLetterExchange") DirectExchange dlx,
            MessagingProperties props) {
        return BindingBuilder.bind(dlq).to(dlx)
                .with(props.queues().tickets().deadLetterKey());
    }
}
```

> "La configuración de mensajería está aislada en el paquete config: un archivo por caso de uso con su cola, su DLQ y sus bindings."

4. Consumidores por dominio · 20 s

Muestra: `tickets/TicketConsumer.java`

```
@RabbitListener(queues = "${pedidos360.messaging.queues.tickets.name}")
public void onMessage(Message message, Channel channel) {
    processor.process(NAME, message, channel,
            PedidoCreadoEvent.class, service::generar);
}
```

> "Hay un paquete por dominio: notificaciones, tickets y documentos. El consumidor solo recibe el mensaje; la lógica de negocio está en `TicketService`, que no conoce RabbitMQ."

5. ACK y manejo de errores · 40 s · el bloque más importante de la pauta

Muestra: `messaging/AckingMessageProcessor.java`, con `acknowledge-mode: manual` y `prefetch: 1`.

```
while (true) {
    attempt++;
    try {
        handler.accept(payload);
        channel.basicAck(tag, false);              // éxito
        return;
    } catch (PermanentProcessingException ex) {
        channel.basicNack(tag, false, false);      // permanente -> DLQ
        return;
    } catch (RuntimeException ex) {                // recuperable
        if (attempt >= retry.maxAttempts()) {
            channel.basicNack(tag, false, false);  // agotado -> DLQ
            return;
        }
        sleeper.sleep(delay);                      // espera 1 s, 2 s
        delay = Math.min((long) (delay * retry.multiplier()),
                retry.maxIntervalMs());
    }
}
```

> "Cada consumidor confirma de forma explícita. Si el procesamiento sale bien hace ACK. Si el error es permanente, como un JSON inválido, hace NACK sin reencolar y el mensaje va directo a la DLQ. Si es recuperable, reintenta con espera creciente y, al agotar los tres intentos, lo manda a la DLQ. Nunca reencolamos en bucle, y un mensaje duplicado se confirma sin reprocesarlo."

6. Microservicio administrador · 30 s

Endpoints: `POST /api/queues`, `GET` y `DELETE /api/queues/{name}`, `POST` y `DELETE /api/exchanges`, `POST` y `DELETE /api/bindings`, `GET /api/monitoring/queues`. Muestra: `dto/QueueCreateRequest.java`

```
public record QueueCreateRequest(
        @NotBlank(message = "name es obligatorio")
        @Pattern(regexp = ValidationPatterns.BROKER_NAME,
                 message = ValidationPatterns.BROKER_NAME_MESSAGE)
        String name,
        Boolean durable,
        Boolean autoDelete,
        String deadLetterExchange,
        String deadLetterRoutingKey,
        @PositiveOrZero(message = "messageTtlMs no puede ser negativo")
        Long messageTtlMs,
        @Positive(message = "maxLength debe ser mayor que 0")
        Long maxLength) { }
```

> "Toda la lógica de administración está en `RabbitAdminService`; los controladores solo llaman a sus métodos. Las entradas se validan con DTO: no se puede crear una cola con nombre vacío, con caracteres inválidos ni con el prefijo reservado `amq.`. Los errores vuelven en JSON con 400, 404 o 409, y modificar RabbitMQ exige el rol Admin."

7. Pruebas · 10 s

> "El proyecto tiene 36 pruebas automáticas: el flujo de ACK, NACK y reintentos, las validaciones y la seguridad del administrador, y el publicador del backend."

8. Cierre · 15 s

> "En resumen, Pedidos360 ahora procesa notificaciones, tickets y documentos de forma asíncrona con RabbitMQ en un clúster de dos nodos, con reintentos, DLQ y monitoreo, sin afectar la lógica que ya existía. Muchas gracias."

Revisión antes de enviar (todos)

☐ El video completo no pasa de 10:00 (Parte 2 como máximo 3:00).
☐ La voz se entiende de principio a fin, sin ruido de fondo.
☐ El diagrama o el código se leen en el video (zoom si hace falta).
☐ No aparecen chats, correos, contraseñas ni otras pestañas.
☐ Se usan los nombres técnicos tal como están: exchange, binding, DLX, DLQ, ACK, NACK, JWT.
☐ Cada parte termina con su frase de traspaso o de cierre.
☐ El archivo tiene el nombre del reparto y ya se lo enviaron a Sebastián.

Preguntas probables del docente

La pauta pide que cada integrante pueda fundamentar el diseño: los tres deben leer esta tabla.

Pregunta	Respuesta corta
¿Diferencia entre DLX y DLQ?	El DLX es un exchange que enruta los mensajes rechazados; la DLQ es la cola donde quedan guardados para revisarlos.
¿Por qué ACK manual?	Para que el consumidor decida el destino de cada mensaje: ACK si salió bien, reintento si es transitorio, NACK sin reencolar hacia la DLQ si no tiene arreglo.
¿Cómo evitan un bucle infinito de reintentos?	Máximo 3 intentos con espera de 1 s y 2 s; después NACK con `requeue=false` y el mensaje sale de la cola principal.
¿Por qué un exchange topic y uno direct?	Topic para eventos que varias colas escuchan por patrón (`pedido.*`); direct para comandos con un solo destinatario por coincidencia exacta (`documento.generar`).
¿Qué pasa si RabbitMQ se cae?	El pedido se guarda igual en la base de datos; el publicador registra el error en el log y no afecta la respuesta al usuario.
¿Las colas están replicadas en los dos nodos?	No: son colas clásicas y viven en un nodo. Para alta disponibilidad real usaríamos quorum queues con 3 nodos; con 2, perder uno detiene la cola.
¿Qué retención tienen las DLQ?	24 horas (`x-message-ttl`) y máximo 10.000 mensajes (`x-max-length`).
¿Cómo saben por qué un mensaje llegó a la DLQ?	Por el log del procesador (`NACK -> DLQ ... motivo=`) y por el header `x-death` del mensaje, que dice la cola de origen y la razón.
¿Y si un mensaje llega dos veces?	El procesador recuerda el `messageId` de los ya procesados y confirma el duplicado sin repetir el trabajo.

¿Dónde se valida el JWT?	Dos veces: en el autorizador JWT del API Gateway (emisor y audiencia) y en cada microservicio con Spring Security (firma, emisor, audiencia, vigencia y roles).
¿Para qué `@ConfigurationProperties`?	Lee los nombres del application.yml en un objeto tipado, así ningún nombre queda escrito a mano en el código.
¿Cómo simulan los errores?	Con marcadores en la descripción del pedido (`#FAIL-TICKET`, `#INVALID-NOTIFICACION`, `#FLAKY-TICKET`), activados solo por configuración para la demostración.